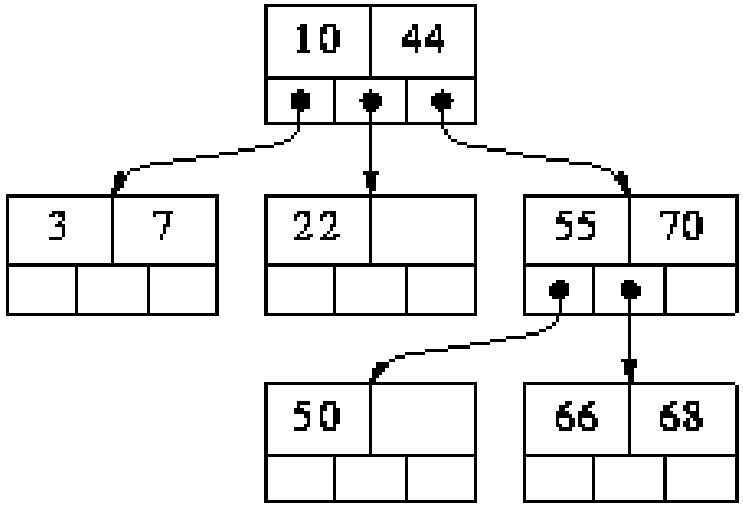
2-4 Trees

AIL 7002

Shweta Agrawal, Amit Kumar, Dr. Ilyas Cicekli, Naveen Garg

Multi-Way Trees

- A binary search tree:
  - *One* value in each node
  - At most 2 children

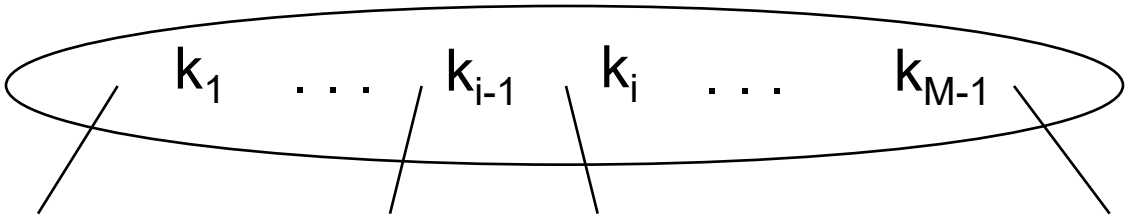


- An *M-way* search tree:
  - Between *1* to *(M-1)* values in each node
  - At most *M* children per node

M-way Search Tree Details

Each internal node of an *M-way* search has:

- Between *1* and *M* children
- Up to *M-1* keys k_1 , k_2 , ... , k_{M-1}

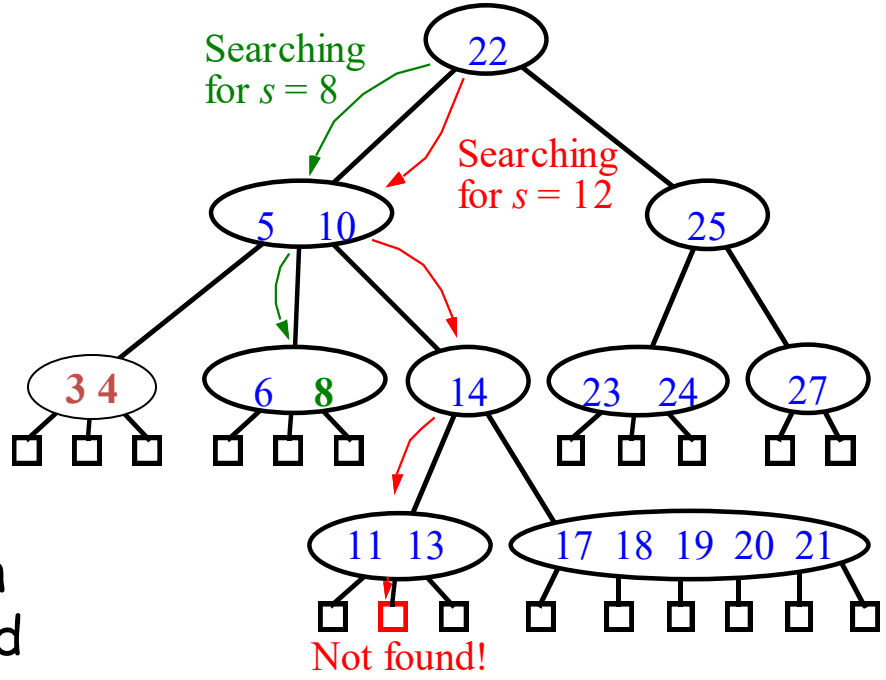


Keys are ordered such that:

$k_1 < k_2 < \ldots < k_{M-1}$

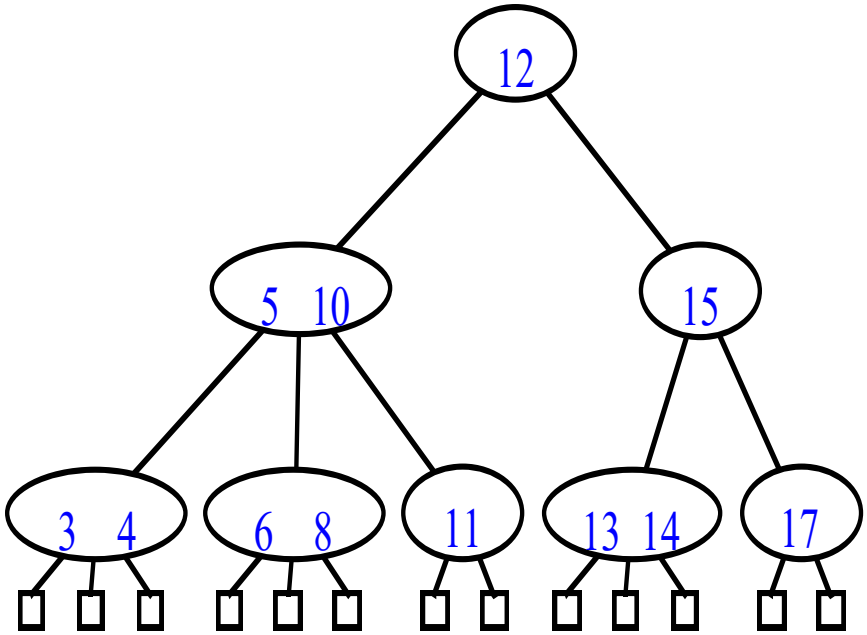
Multi-way Searching

- Similar to binary searching
  - If search key $s<k_1$ search the leftmost child
  - If $s>k_{d-1}$, search the rightmost child
- Multiway search tree ?
  - Find two keys k_{i-1} and k_i between which s falls, and search the child v_i.
- What would an in-order traversal look like?



(2,4) Trees

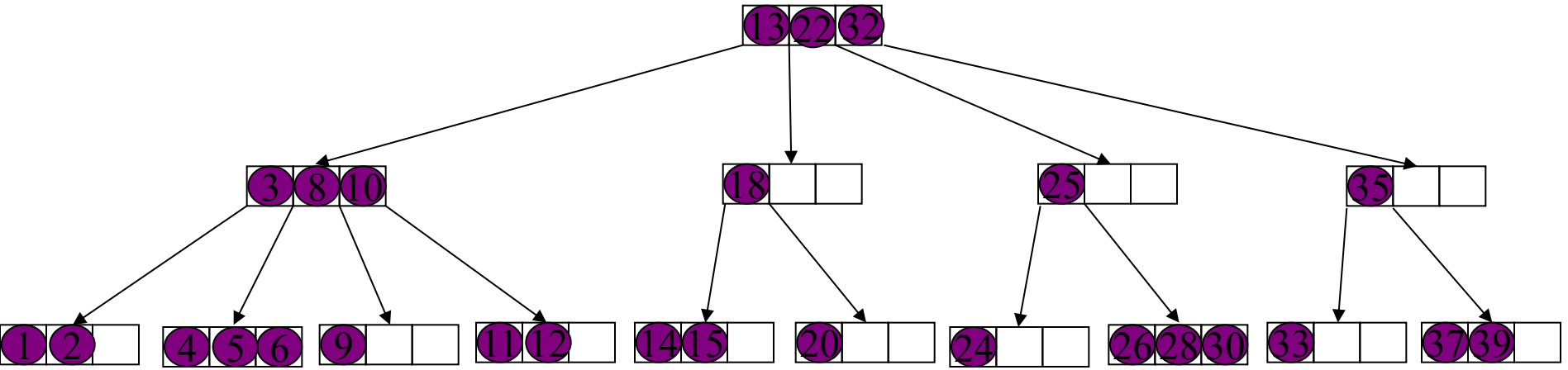
- Properties:
  - Each node has at most 4 children
  - All external nodes have same depth
  - Height *h* of (2,4) tree is *O*(log *n*).
- How is the last fact useful in searching?



Insertion

- No problem if the node has empty space

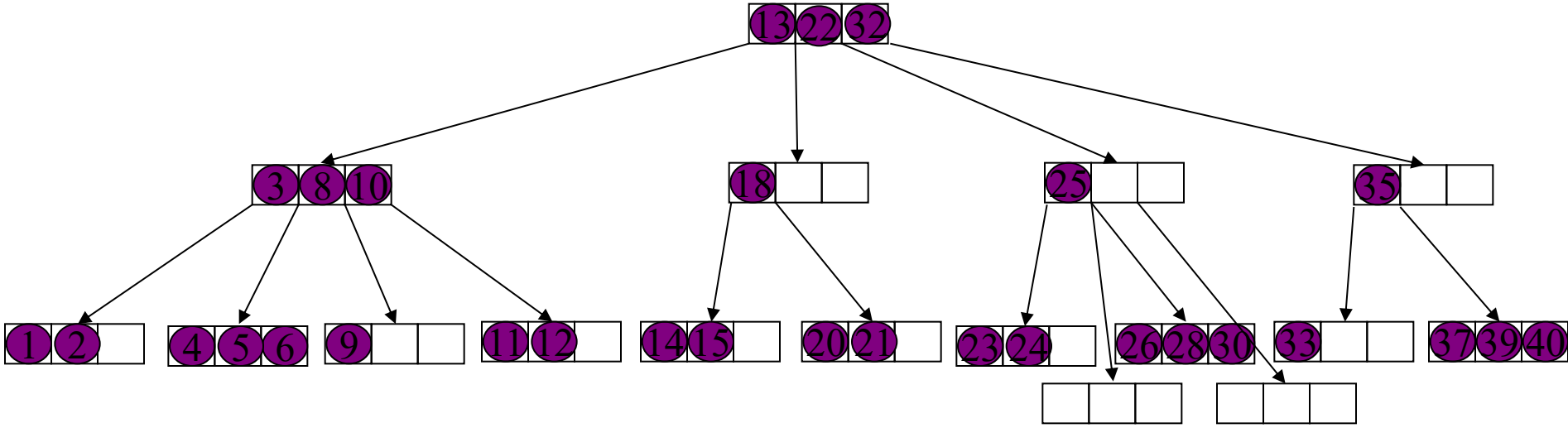
21 23 40 29 7



Insertion(2)

- Nodes get split if there is insufficient space.

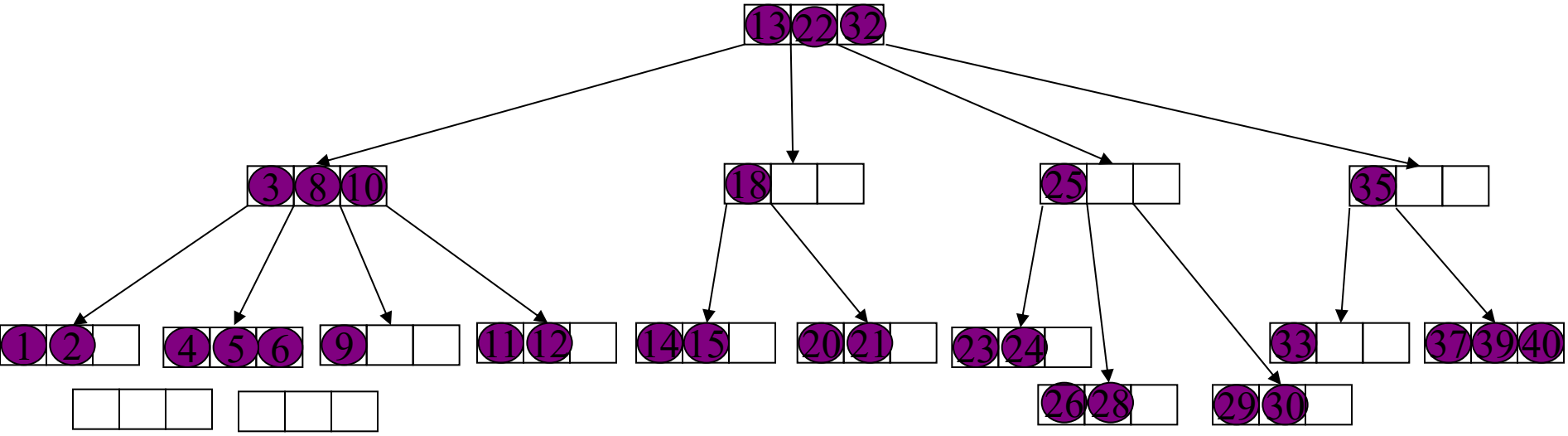
29 7



Insertion(3)

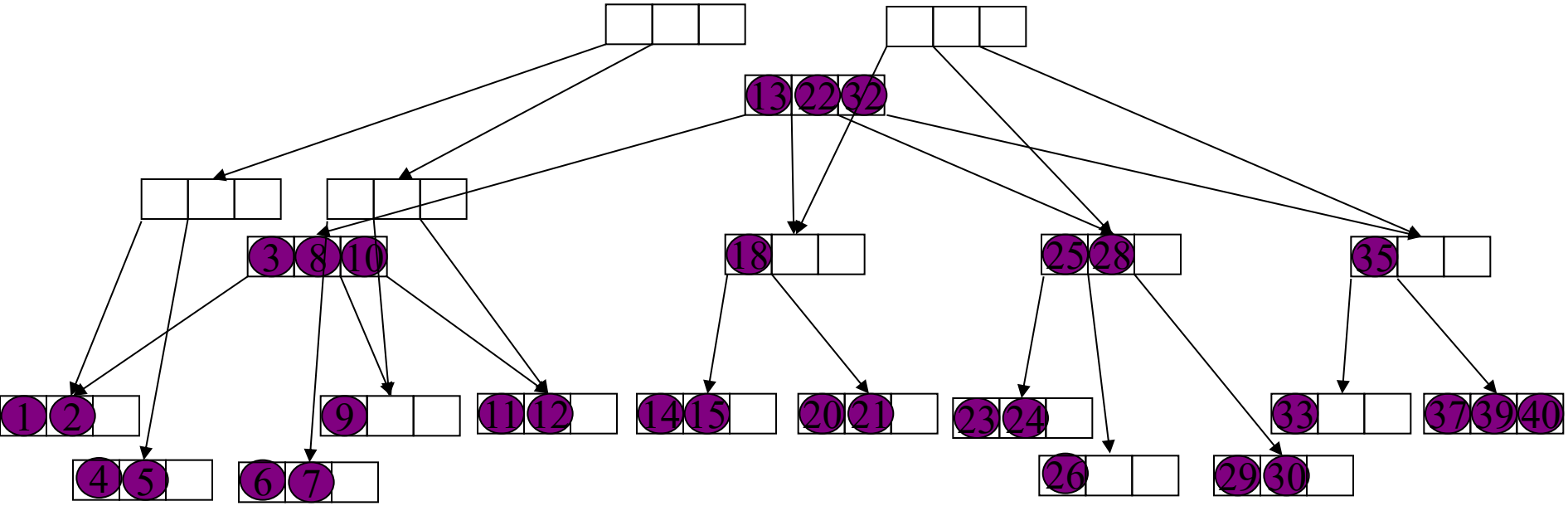
7

- One key is promoted to parent and inserted in there



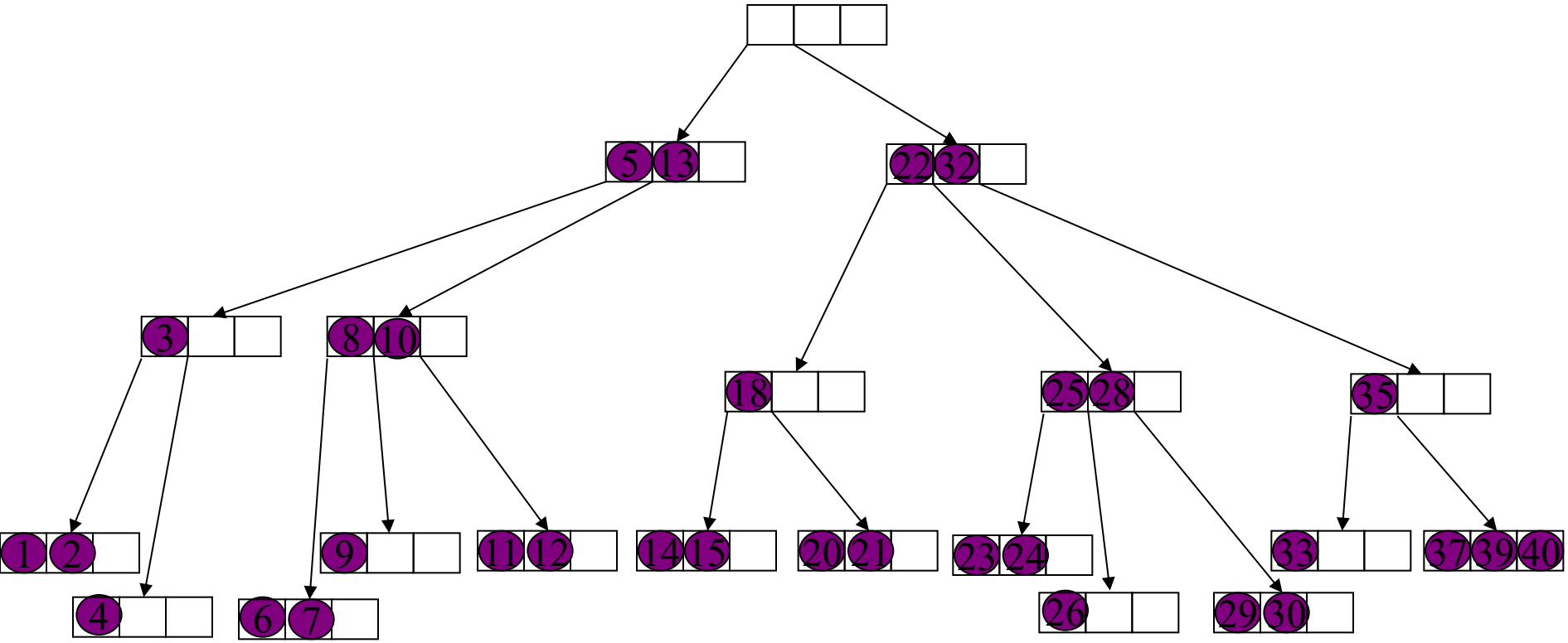
Insertion(4)

- If parent node does not have sufficient space then it is split.
- In this manner splits can cascade.



Insertion(5)

- Eventually we may have to create a new root.
- This increases the height of the tree

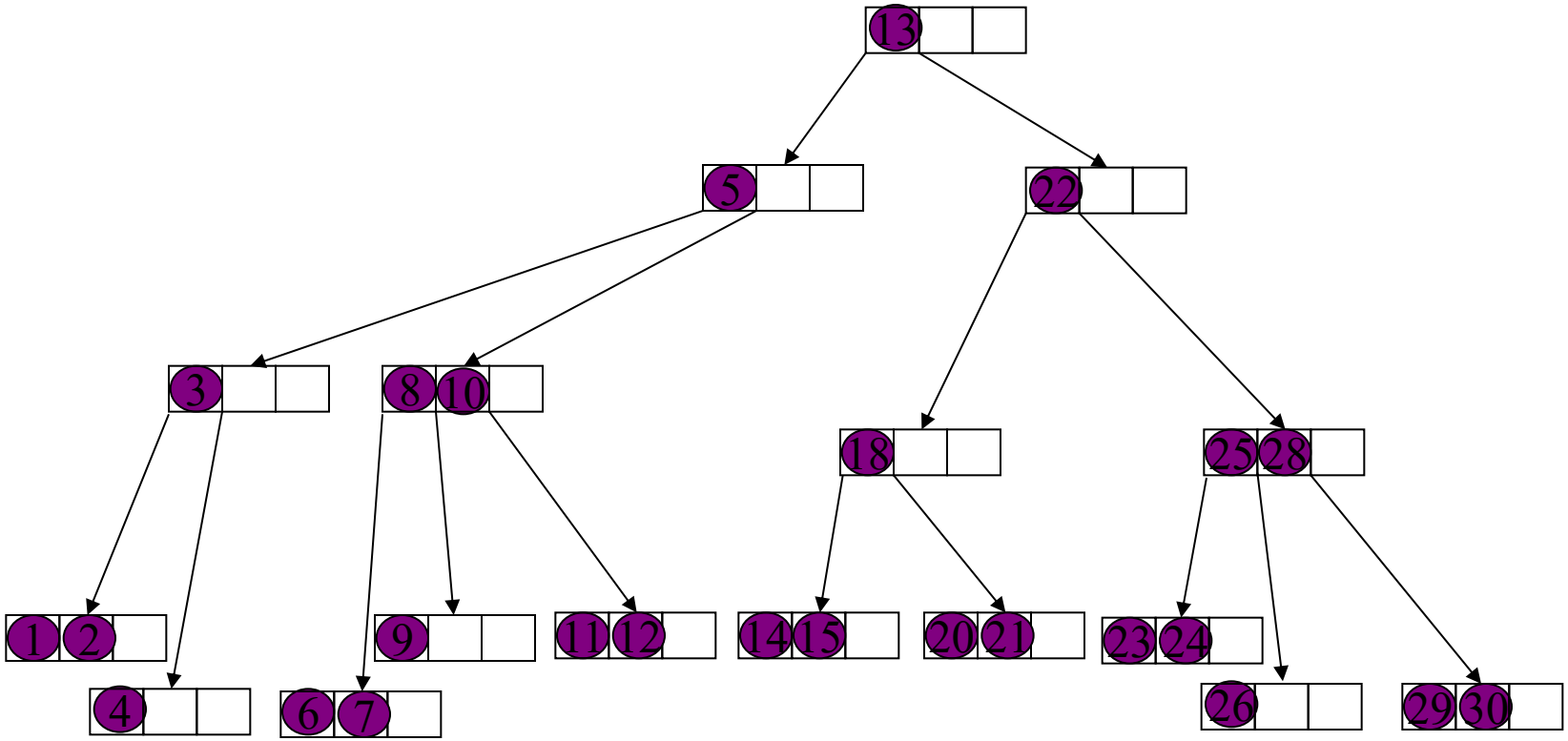


Time for Search and Insertion

- A search visits O(log N) nodes
- An insertion requires O(log N) node splits
- Each node split takes constant time
- Hence, operations Search and Insert each take time O(log N)

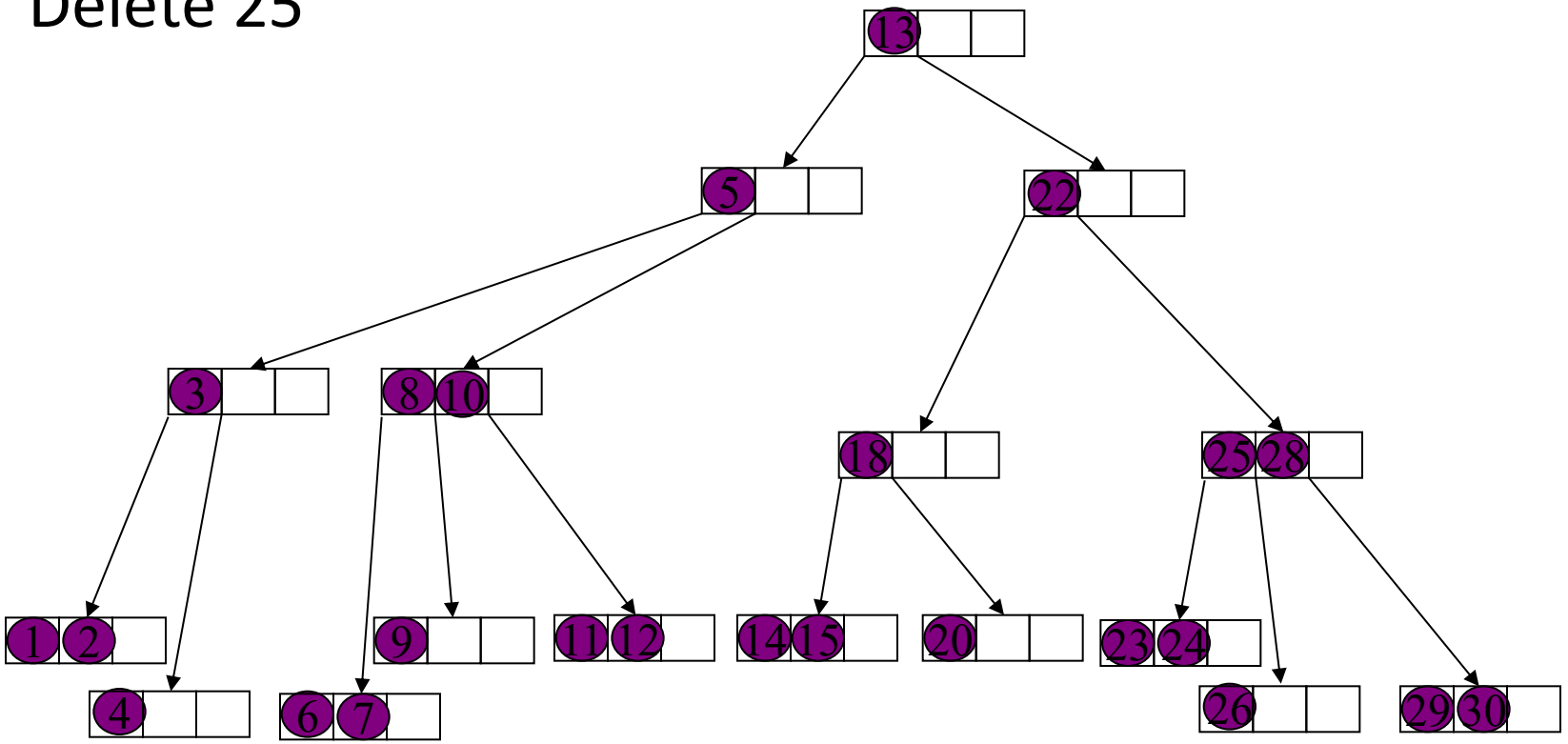
Deletion

- Delete 21.
- No problem if key to be deleted is in a leaf with at least 2 keys



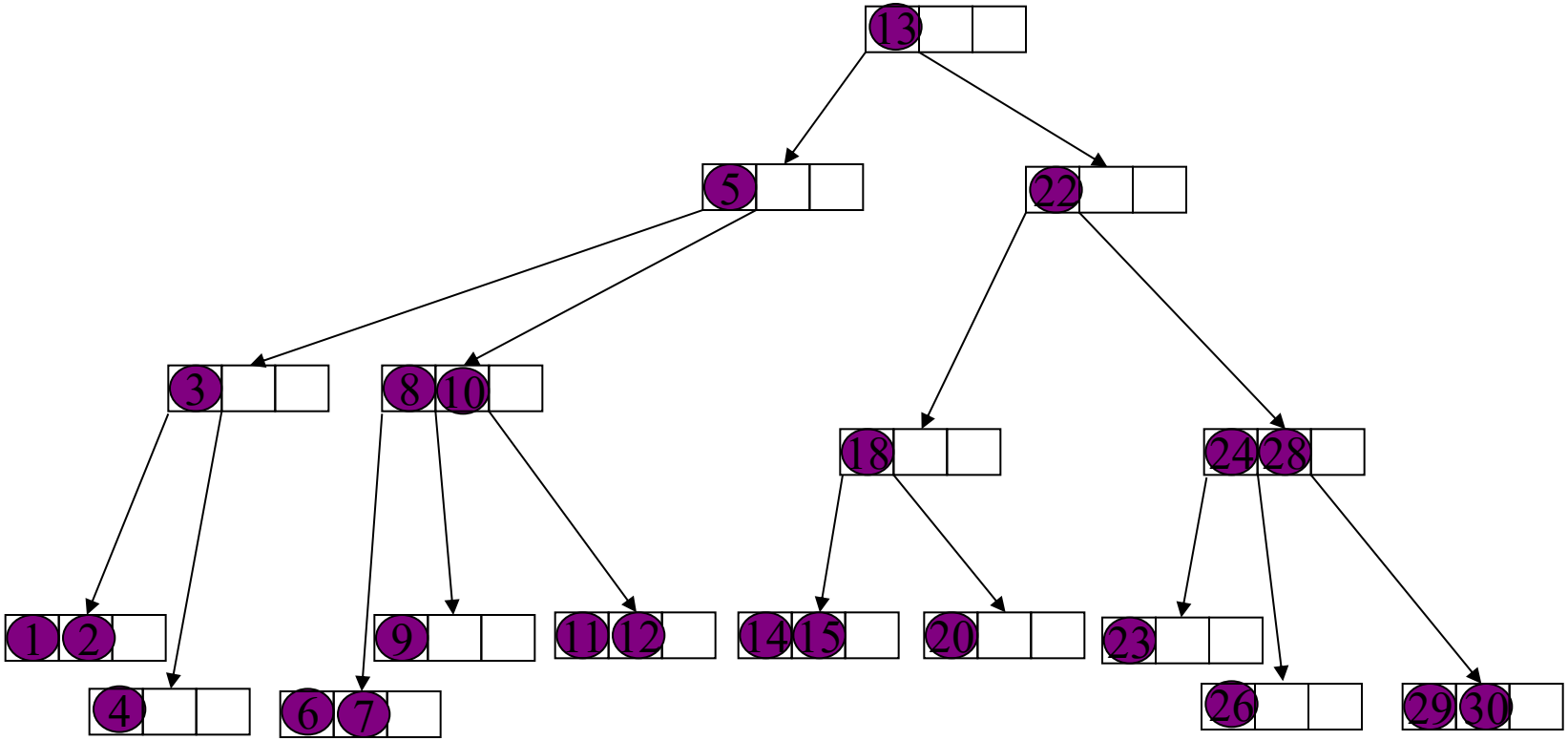
Deletion(2)

- If key to be deleted is in an internal node then we swap it with its predecessor (which is in a leaf) and then delete it.
- Delete 25



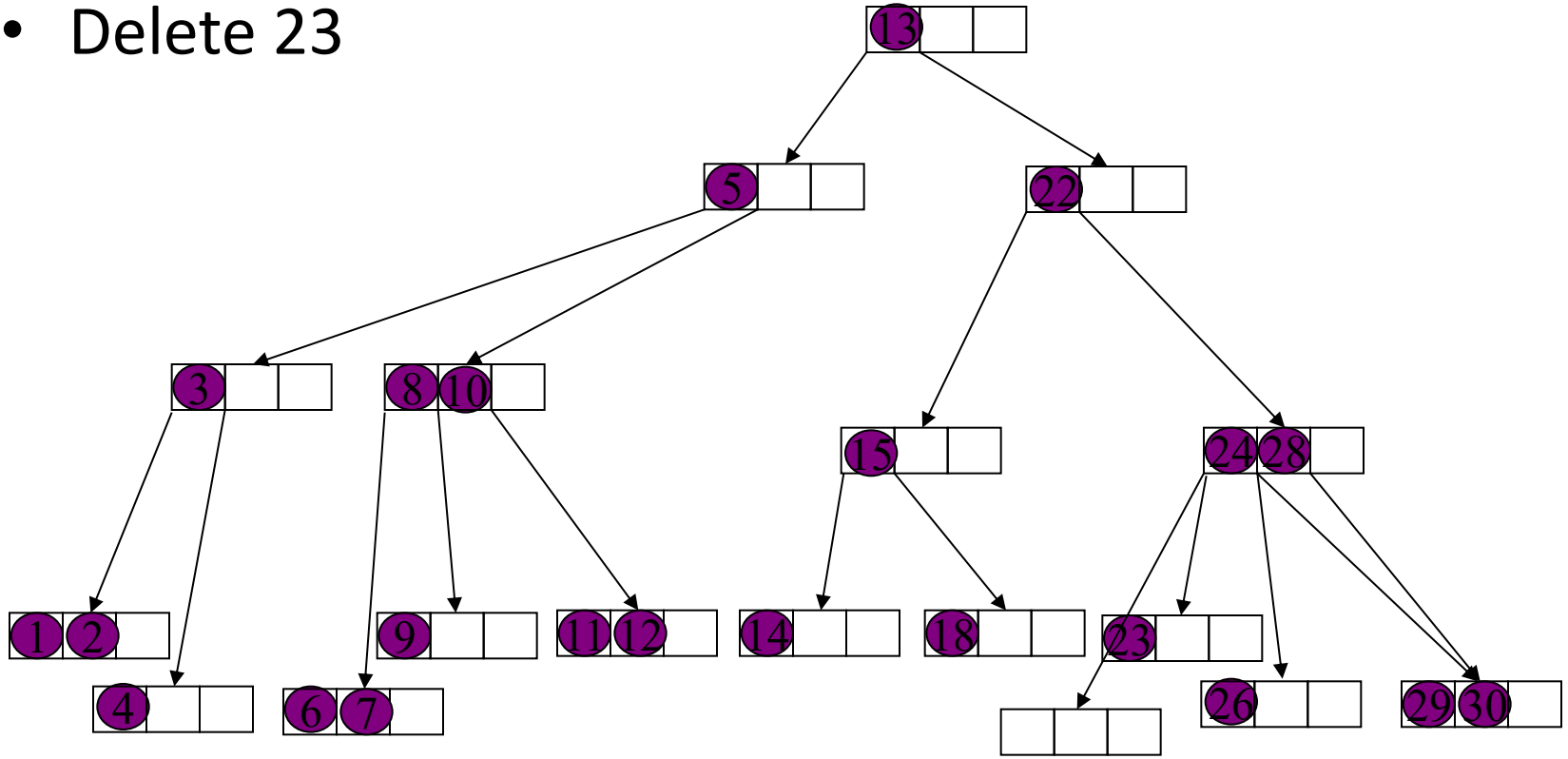
Deletion(3)

- If after deleting a key a node becomes empty then we borrow a key from its sibling.
- Delete 20



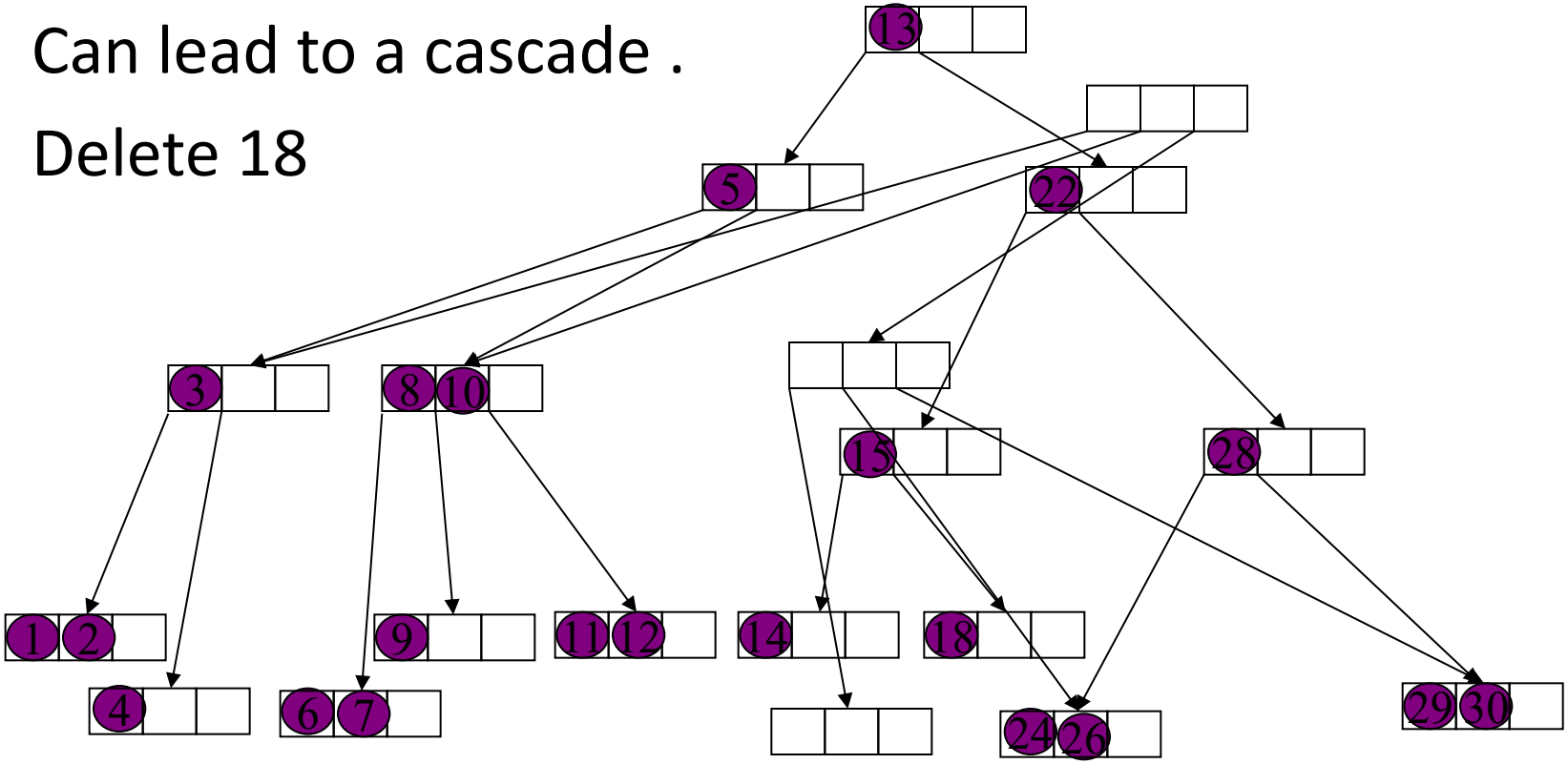
Deletion(4)

- If sibling has only one key then we merge with it.
- The key in the parent node separating these two siblings moves down into the merged node.
- Delete 23



Delete(5)

- Moving a key down from the parent corresponds to deletion in the parent node.
- The procedure is the same as for a leaf node.
- Can lead to a cascade .
- Delete 18

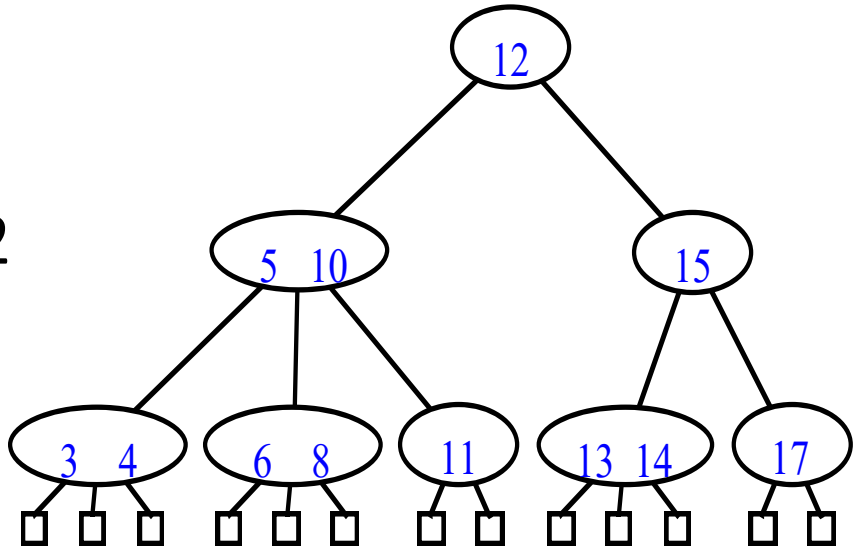


(2,4) Conclusion

- The height of a (2,4) tree is $O(\log n)$.
- Split, transfer, and merge each take $O(1)$.
- Search, insertion and deletion each take $O(\log n)$.
- Why are we doing this?
  - (2,4) trees are fun! Why else would we do it?
  - Well, there's another reason, too.
  - They can be extended to what are called B-trees.

(a,b) Trees

- A multiway search tree.
- Each node has at least *a* and at most *b* children.
- Root can have less than *a* children but it has at least 2 children.
- All leaf nodes are at the same level.
- Height *h* of (a,b) tree is at least $\log_b n$ and at most $\log_a n$.

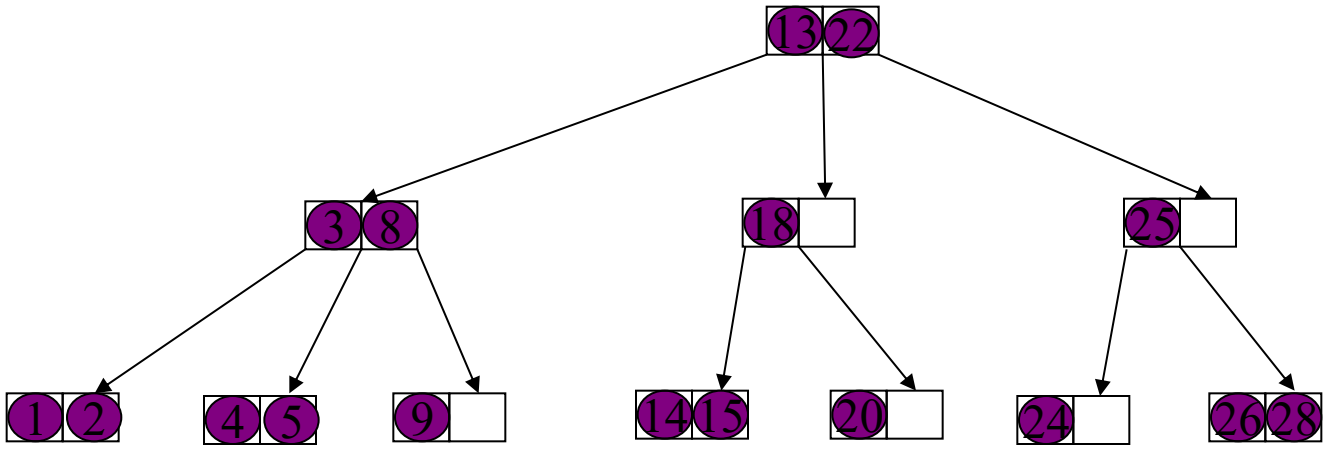


Insertion

- No problem if the node has empty space

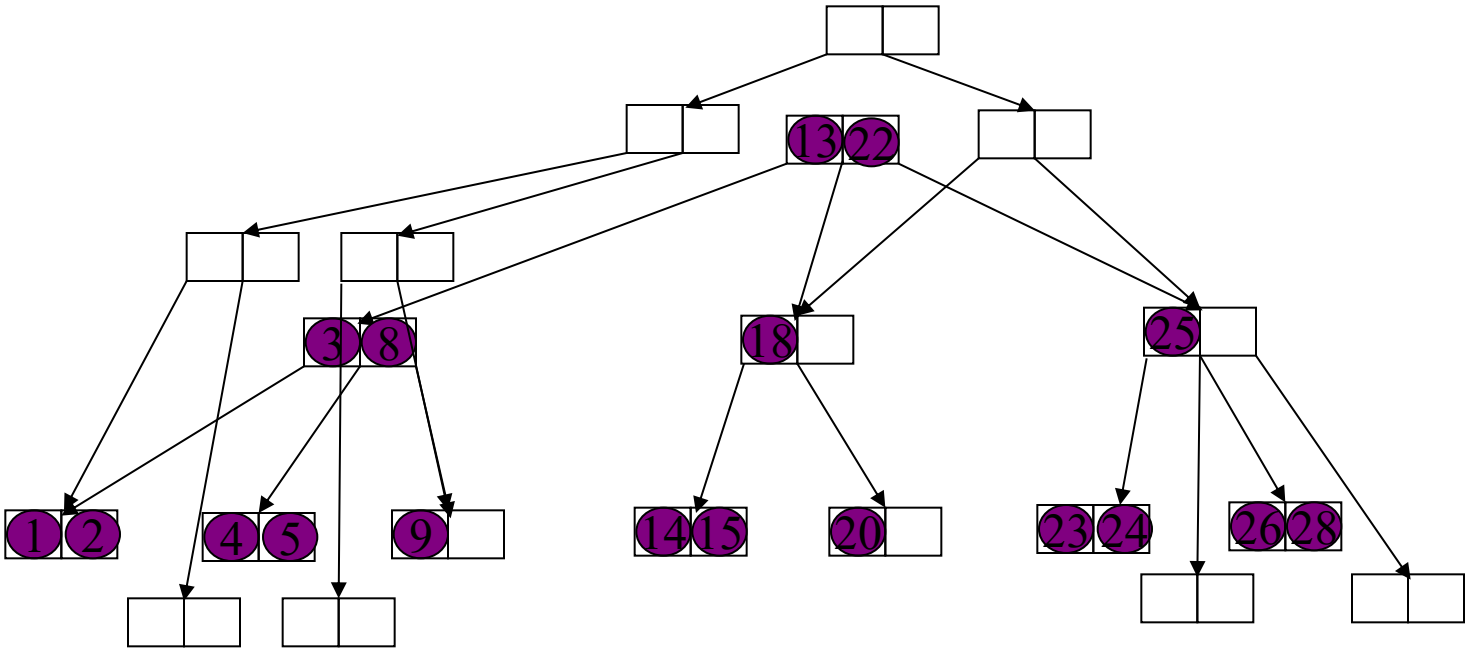
21 23 29 7

$b = 3$



Insertion(2)

- Nodes get split if there is insufficient space.
- The median key is promoted to the parent node and inserted there

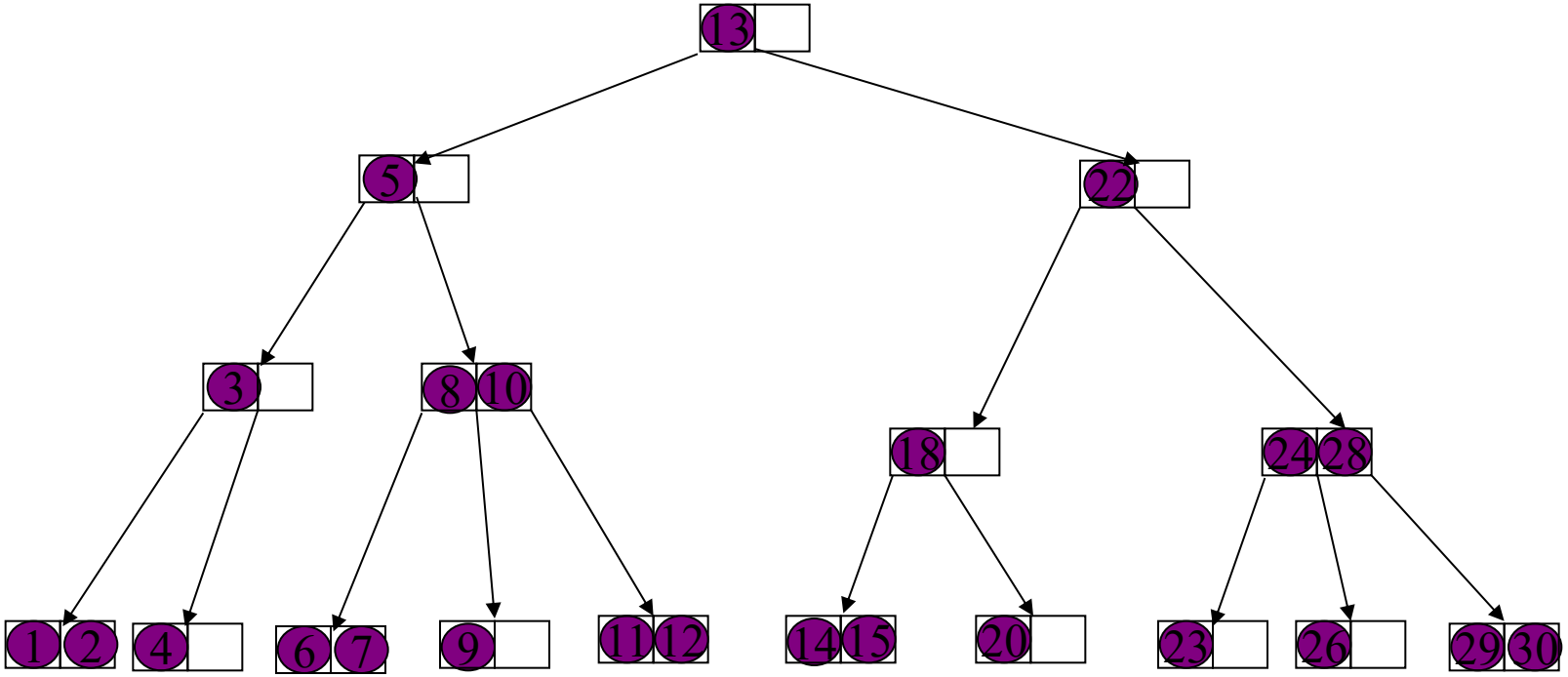


Insertion(3)

- A node is split when it has exactly *b* keys.
- One of these is promoted to the parent and the remaining are split between two nodes.
- Thus one node gets $\lceil \frac{b-1}{2} \rceil$ and the other $\lfloor \frac{b-1}{2} \rfloor$ keys.
- This implies that a-1 <= $\lfloor \frac{b-1}{2} \rfloor$

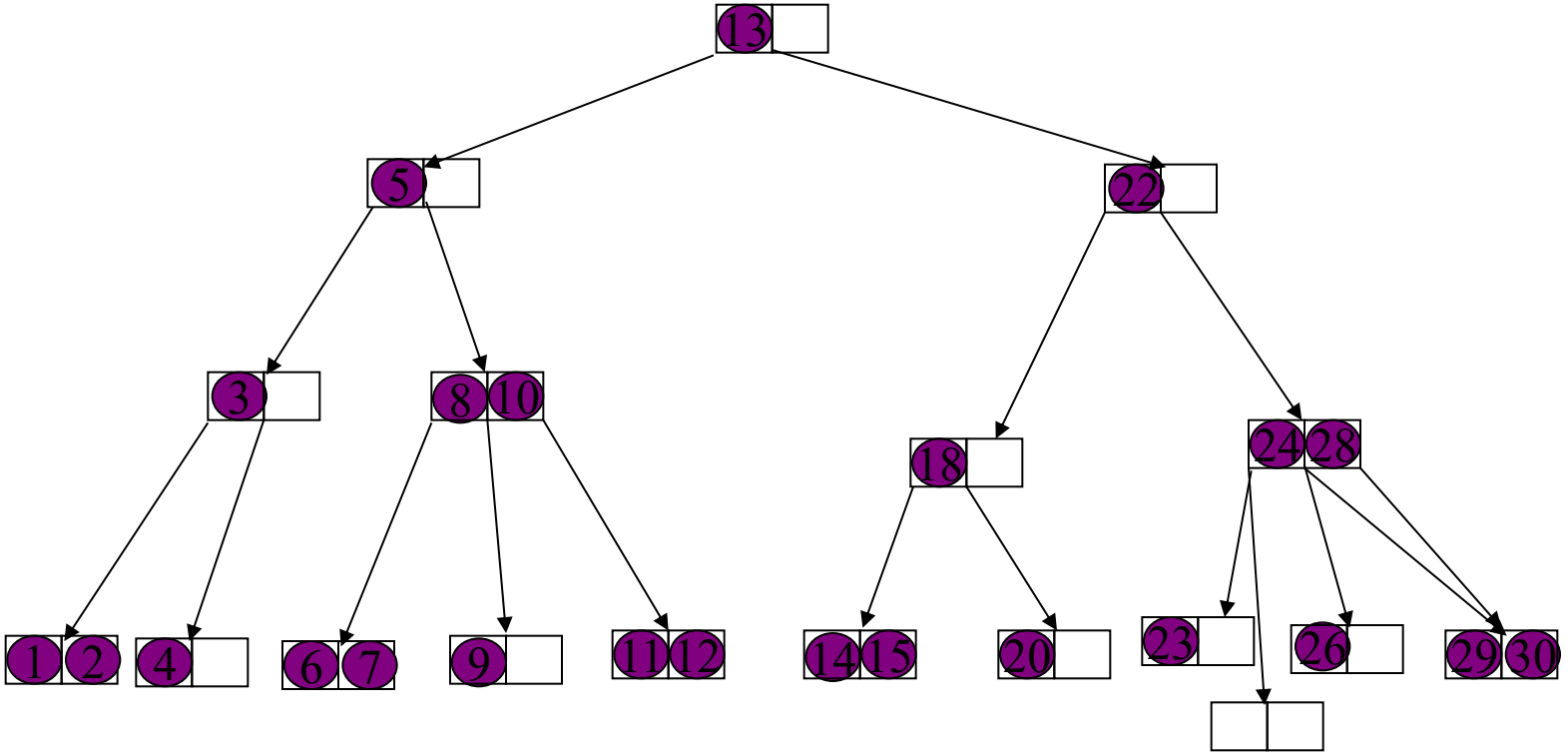
Deletion

- If after deleting a key a node becomes empty then we borrow a key from its sibling.
- Delete 20



Deletion(2)

- If sibling has only one key then we merge with it.
- The key in the parent node separating these two siblings moves down into the merged node.
- Delete 23



Deletion(3)

- In an (a,b) tree we will merge a node with its sibling if the node has *a*-2 keys and its sibling has *a*-1 keys.
- Thus the merged node has 2(*a*-1) keys.
- This implies that 2(*a*-1) <= *b*-1 which is equivalent to *a*-1 <= $\lfloor \frac{b-1}{2} \rfloor$
- Earlier too we argued that *a*-1 <= $\lfloor \frac{b-1}{2} \rfloor$
- This implies *b* >= 2*a*-1
- For a=2, *b* >= 3

Conclusion

- The height of a (a,b) tree is $O(\log n)$.
- $b >= 2a-1$.
- For insertion and deletion we take time proportional to the height.